Final Project: Autonomous Drone Planning for Smart-City Emergency Response

Course: Planning and Approximate Reasoning (MIA-MESIIA) **Group size:** 1 or 2 students **Submission deadline:** 6 November 2026 at 23:59 Europe/Madrid time (25% of the course grade) **Second submission deadline:** 17 January 2027 at 23:59 Europe/Madrid time, maximum grade 8 **Submission:** Moodle — one ZIP file per group

The ZIP file must be named: NameSurnamesStudent1_NameSurnamesStudent2_PAR_Final.zip (if you work alone: NameSurnames_PAR_Final.zip)

It must contain all PDDL source files and a PDF report. Submit the PDDL files as plain-text .pddl files (exactly the files you give to the planner), with comments, and a README.txt with the commands, so that the grader can run them.

---

1. Introduction

Autonomous drones are increasingly used in emergency-response scenarios: delivering medical supplies to disaster sites, inspecting damaged infrastructure, and reaching locations that are inaccessible to ground vehicles. Planning the actions of a drone fleet under real-world constraints — limited battery, ordered mission goals, and scarce charging infrastructure — is a challenging and practically relevant problem.

In this project you will design, implement, and evaluate a PDDL-based planning system for an autonomous drone emergency-response scenario. You will model the domain in PDDL, design three problem instances of increasing difficulty, run a planner, validate your results, and reflect critically on your modeling choices and experimental findings.

---

2. Scenario

A smart-city control center has deployed a fleet of autonomous drones following a local emergency. Several city locations require inspection — some may be unsafe or inaccessible until neighboring areas have been cleared. Other locations are waiting for emergency supplies such as medical kits, water packages, or communication devices, which must be collected from a central depot and delivered to their destinations.

Drones have limited battery. Every flight between locations consumes battery proportional to the distance traveled. When battery runs low, a drone must navigate to a charging station before continuing its mission. Charging stations are fixed at specific points in the city and are not always conveniently placed.

The planner must find a valid sequence of drone actions that completes all required inspections and deliveries while respecting battery limits, supply availability, and any ordering constraints imposed by the scenario.

Illustrative example. Figure 1 shows a small example that is not one of the three instances of Section 5 (two drones, six locations, battery capacity 5). It only shows how a plan changes the state: each flight lowers the battery by its flight cost, a recharge refills it (Model A, Section 4), an inspection can unlock a location, and every action adds to the plan cost. Figure 2 gives an optimal plan for it under Model A (cost 10, checked with A* + LM-cut and with VAL); without the recharge action the example has no solution. Your own instances must follow the specifications of Section 5.

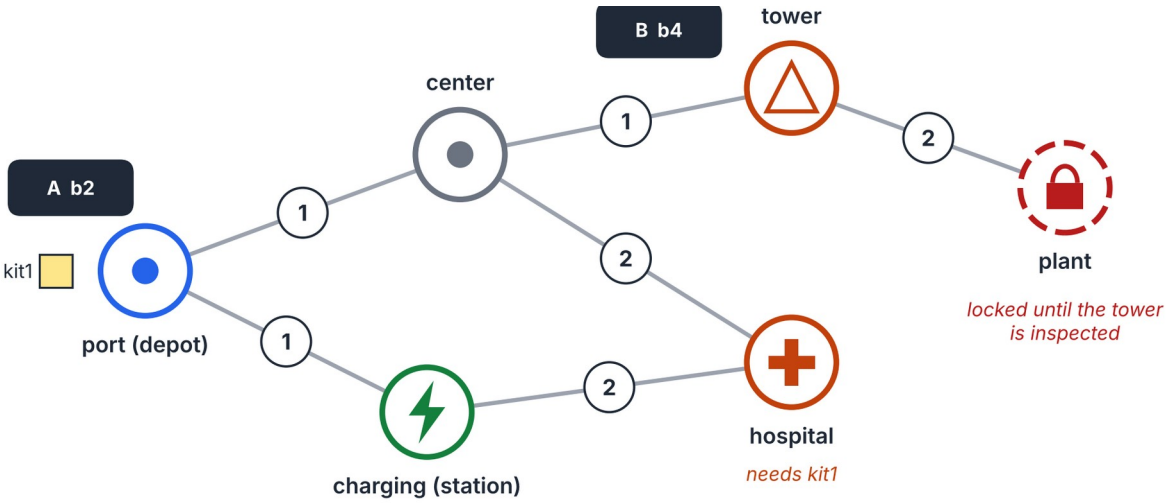


Figure 1. *Illustrative example (not one of the project instances): map and initial state.*

#	Action	A at	A batt.	A carries	B at	B batt.	cost	Note
0	initial state	port	b2	nothing	tower	b4	**0**	
1	`(pick-up-supply droneA kit1 port)`	port	b2	**kit1**	tower	b4	**1**	
2	`(inspect droneB tower)`	port	b2	kit1	tower	b4	**2**	plant unlocked
3	`(fly droneA port charging)`	**charging**	**b1**	kit1	tower	b4	**3**	flight cost 1
4	`(fly droneB tower plant)`	charging	b1	kit1	**plant**	**b2**	**5**	flight cost 2
5	`(recharge droneA charging)`	charging	**b5**	kit1	plant	b2	**6**	b1 < 2: must recharge
6	`(inspect droneB plant)`	charging	b5	kit1	plant	b2	**7**	
7	`(fly droneA charging hospital)`	**hospital**	**b3**	kit1	plant	b2	**9**	flight cost 2
8	`(deliver-supply droneA kit1 hospital)`	hospital	b3	**nothing**	plant	b2	**10**	all goals reached

Cost: each flight adds its flight cost; every other action adds 1. Blue = value changed by that step. Only drone and location arguments are shown; in your plan files the actions also carry the level arguments of your own encoding.

Figure 2. *An optimal plan for the example of Figure 1 under Model A (recharge to the maximum level).*

---

3. PDDL Domain Requirements

3.1 Types

Your domain must define at least the following types:

```
(:types
  drone
  location
  supply
  level      ; discrete battery levels (see Section 3.2)
)
```

You may define subtypes if your model requires them. Every type must be explained in your report. Careful: in a :types list, “- T” applies to every name written before it since the previous dash, also across line breaks. To declare kinds of supply, write drone location supply level - object on one line and then, for example, medkit water - supply. Without “- object”, drone, location and level become subtypes of supply: Fast Downward accepts this silently, but VAL’s Parser tool (used by some editors for syntax checks) never finishes.

3.2 Symbolic Battery

Fast Downward, the planner used in this project (Section 7), does not support numeric fluents: it cannot handle numeric conditions such as (>= (battery-level ?d) ...) or numeric effects such as (decrease ...) on state variables. Battery must therefore be modeled **symbolically, using discrete battery-level objects**. A drone cannot fly if its current battery level is below the cost of the intended flight. Recharging at a charging station raises the battery level; how much depends on the model (Section 4).

A recommended encoding is shown below. You may rename predicates or use a different symbolic encoding, provided you explain it in your report:

```
(:predicates
  (battery ?d - drone ?l - level)              ; current battery level of drone ?d
  (flight-cost ?from ?to - location ?c - level)  ; flying ?from → ?to costs ?c levels
  (battery-after ?before ?c ?left - level)      ; ?before - ?c = ?left (listed only if ?before >= ?c)
  (next-level ?l1 ?l2 - level)                  ; ?l2 = ?l1 + 1 (used by Model B's partial recharge)
  (max-level ?l - level)                        ; full battery capacity
)
```

The level objects (e.g. b0, b1, ..., b6) and all battery-after, next-level, max-level and flight-cost facts are declared in each problem file. Because battery-after is only listed when the result is not negative, a flight with insufficient battery has no matching fact and therefore cannot be applied.

Plan cost. Use the **:action-costs** requirement so that plan cost is well defined. This is the only numeric construct allowed, and Fast Downward supports it. Declare (total-cost) and a static function (distance ?from ?to - location); let fly increase total-cost by the distance of the flight (equal to its flight cost) and every other action by 1; and add (:metric minimize (total-cost)) to every problem file. In :init, set (= (total-cost) 0) and give every (= (distance ?a ?b) k) value.

```
(:functions
  (total-cost) - number
  (distance ?from ?to - location) - number   ; static: never changed by any action
)
```

3.3 Required Actions

Your domain must include at least the following five actions. You may add further actions if needed.

Requirements. Declare in the domain every requirement you use, for example (:requirements :strips :typing :negative-preconditions :conditional-effects :action-costs). Negative effects such as (not (at ?d ?from)) are delete effects: they are part of STRIPS and need no extra requirement. A negative precondition, such as (not (carrying ?d ?s)) or (not (inspected ?l)) in a :precondition, needs :negative-preconditions. :conditional-effects is needed only for forall/when effects (see the unlocking hint below), and :action-costs for total-cost. Fast Downward and VAL support all of them; numeric fluents remain forbidden (Section 3.2).

---

fly — A drone moves from one location to another. The two locations must be connected and the destination must be accessible. The drone must have sufficient battery. The battery level decreases by the flight cost.

```
(:action fly
  :parameters (?d - drone ?from ?to - location ?before ?c ?left - level)
  :precondition (and
    (at ?d ?from)
    (connected ?from ?to)
    (accessible ?to)                ; locked locations cannot be entered
    (battery ?d ?before)
    (flight-cost ?from ?to ?c)
    (battery-after ?before ?c ?left)   ; only exists if ?before >= ?c
  )
  :effect (and
    (not (at ?d ?from))
    (at ?d ?to)
    (not (battery ?d ?before))
```

```
    (battery ?d ?left)
    (increase (total-cost) (distance ?from ?to))
  )
)
```

This fly action is a template. Its (accessible ?to) precondition is needed for Instances 2 and 3 (Section 5), where some locations stay locked until another location has been inspected or a supply has been delivered. You may, and are expected to, extend fly and the other actions as your model requires. Every location that is open from the start, including the start location of each drone, must be declared (accessible ...) in :init.

Flight cost and distance. Every connected pair needs both a flight-cost fact (battery levels) and a distance value (plan cost), with the same number and in both directions, for example (flight-cost a b b2) and (= (distance a b) 2). A missing or different value is a common bug: check both lists against your location diagram.

Parameter names. Do not call a parameter ?after: VAL treats after as a keyword and rejects the whole domain (“Problem in domain definition!”). The templates above use ?left.

---

inspect — A drone inspects a location. The drone must be present at that location. Inspecting a location may make other locations accessible.

Hint (unlocking). One simple encoding uses a static predicate in the problem file, for example (unlocked-by-inspection ?x ?y - location), and a conditional effect in inspect:

(forall (?y - location) (when (unlocked-by-inspection ?l ?y) (accessible ?y)))

Add :conditional-effects to :requirements; Fast Downward and VAL support it. The same idea works for deliver-supply. Alternatively, give the location that is unlocked as an extra parameter of the action.

---

pick-up-supply — A drone picks up a supply. The drone and the supply must be at the same location. The drone must not already be carrying a supply.

---

deliver-supply — A drone delivers a supply to a location that needs it. The drone must be carrying the supply and present at the target location. Delivering a supply may make other locations accessible.

Which supply? A location that needs a supply needs one specific supply object, for example (needs site3 kit1), and the goal says that this object has been delivered there. The kind of a supply (medical kit, water, communication device) can be a subtype of supply or a predicate; “different types” in Section 5 means different kinds.

---

recharge — A drone recharges its battery. The drone must be at a location that has a charging station. How far the battery level is raised depends on the model (Section 4).

Every action must have clearly written preconditions and effects in the domain file, be fully commented in the code, and be explained in the report.

3.4 Predicates

You must design your own predicate set. As a minimum, your domain must be able to represent:

- the current location of each drone
- which locations are connected
- which locations have been inspected
- which locations need a specific supply
- where supplies are currently located
- what each drone is carrying
- which locations have a charging station
- which supplies have been delivered
- the current battery level of each drone, and how each flight changes it
- which locations are currently accessible, and which inspection or delivery unlocks each locked location

You may add further predicates as your model requires. Every predicate must be explained in the report.

---

4. Two Alternative Domain Models

You must submit **two PDDL domain files** modeling the same scenario in two meaningfully different ways. Both models must be correct, and every problem instance must be solvable under both models: show this with at least one VAL-validated plan for each instance and model. A particular planner configuration may still fail to find a plan within the time or memory limit; such failures are acceptable if you report and discuss them (Section 7.2).

The difference must reflect a genuine modeling decision. For all groups this decision is the **recharge model**:

Option	Model A	Model B
Recharge	One recharge action sets the battery directly to the maximum level, whatever the current level	Each recharge action raises the battery by one level (next-level); several recharge actions are needed to reach full capacity

If you wish to propose a different alternative, you must obtain the instructor's approval by 28/10/2026. Any alternative must stay compatible with Fast Downward (no numeric fluents).

In your report you must compare the two models on four dimensions:

- **Expressiveness** — which model represents the scenario more realistically and why
- **Planning complexity** — which model is easier for the planner to solve, and what in the model structure explains this
- **Plan quality and runtime** — differences in plan length, plan cost, runtime and expanded states
- **Trade-offs** — what each model gains and what it sacrifices

This comparison must be supported by evidence from your experimental results.

---

5. Problem Instances

You must create three problem instances of increasing difficulty. All three instances must be solvable under both domain models (Section 4).

Instance 1: Small

- One drone
- Five locations: one depot, one charging station, three mission sites
- Battery capacity: 4 levels (b0–b4); all flight costs: 1 level per edge
- Goals: inspect two locations, deliver one supply
- **Design constraint:** the drone must recharge at least once to complete the mission; a solution that avoids recharging must not exist. Justify this in the report (e.g. by showing that the shortest route covering all goals costs more than 4 levels).

Instance 2: Medium

- Two drones
- Eight locations: one depot, two charging stations, five mission sites
- Battery capacity: 6 levels (b0–b6); flight costs vary between 1 and 2 levels per edge
- Goals: inspect four locations, deliver three supplies of at least two different types
- **Design constraint:** at least one mission site cannot be entered until a neighboring location has been inspected. In the report, name an action sequence that would fail (e.g. flying straight to the locked site) and explain which precondition blocks it.

Instance 3: Large

- Three drones
- At least ten locations: one or two depots, multiple charging stations, multiple mission sites
- Multiple supplies of different types
- Goals: at least six inspection and delivery goals combined
- **Design constraint:** at least one supply must be delivered before another location becomes accessible.

- **Single-drone comparison:** also create a single-drone variant (problem-large-1drone.pddl: same map and goals, one drone). Design the large instance so that multiple drones are expected to reduce the total mission cost. Compare the best validated plan costs found for the single-drone and three-drone variants (same domain model and planner configuration), quantify the difference (absolute and in %), and explain it. If the expected advantage does not appear, investigate why. Compare plan cost, not plan length: classical plans are sequential, so the three-drone plan may contain as many actions as the single-drone plan.

Hint: start the three drones in different parts of the map (for example at different depots or charging stations), so that each one serves its own group of mission sites. The single drone must then fly much further and recharge more often. Compare the costs of good plans: LAMA-first often returns plans far from the best one, which can hide the difference. Also run --alias lama (it keeps improving the plan until the time limit) and report the best cost it finds. It writes a new plan file each time it finds a cheaper plan (sas_plan.1, sas_plan.2, ...): validate and report the last one.

For each instance, include a **diagram** in your report showing the location graph with edge costs, depots, charging stations, mission sites, drones, initial supply positions, and the dependencies (which inspection or delivery unlocks which location).

---

6. Plan Validation

All plans you report must be validated using **VAL**, the standard PDDL plan validator. Store the VAL output for every reported plan in the validation/ folder, and include at least one VAL output per instance in the report appendix. Plans that have not been validated will not receive full marks in the empirical evaluation section.

VAL is available at: https://github.com/KCL-Planning/VAL

Run VAL on the same domain and problem files you give the planner, for example: Validate -v domain-A.pddl problem-small.pddl sas_plan. VAL supports :action-costs and reports the plan cost.

---

7. Empirical Evaluation

7.1 Planner and Configurations

Run your experiments using **Fast Downward**. Test at least the following two planner configurations:

- --alias lama-first — LAMA-first, a satisficing planner (landmark and FF heuristics, greedy search)
- --search "astar(lmcut())" — A* with the LM-cut heuristic (cost-optimal)

What these configurations do. LAMA-first is a greedy best-first search guided by the FF heuristic (Lectures 2 and 4) and a landmark heuristic, and it tries FF's helpful actions first: fast, but plans are not optimal. LM-cut is an admissible heuristic: it finds sets of actions of which every relaxed plan must use at least one (action landmarks) and adds up, for each such set, the cost of its cheapest action. With A* it returns cost-optimal plans, but it needs much more time and memory.

You may test additional configurations if you wish, for example --search "eager_greedy([ff()])" (greedy best-first search with the FF heuristic of Lecture 2; note that the Fast Downward syntax is not the pyperplan one).

Use a 5-minute time limit and the same memory limit for every run, for example:

./fast-downward.py --overall-time-limit 5m --overall-memory-limit 4G --alias lama-first domain-A.pddl problem-small.pddl

./fast-downward.py --overall-time-limit 5m --overall-memory-limit 4G domain-A.pddl problem-small.pddl --search "astar(lmcut())"

7.2 Results Table

Report your results for every combination of instance, domain model, and planner configuration:

Instance	Domain model	Planner	Solved	Runtime (s)	Plan length	Plan cost	Expanded states

If a configuration does not solve an instance within a 5-minute time limit, report it as unsolved and note whether the limit was time or memory. An unsolved result is acceptable (it is likely for A* with LM-cut on the larger instances): report it and explain why in Section 7.3. An unsolved run is not the same as an unsolvable instance: every instance must have a plan (Section 4), even if a particular configuration cannot find it within the limits. Plan cost is the final value of total-cost reported by Fast Downward and VAL.

7.3 Analytical Discussion

For each problem instance write one paragraph addressing all of the following:

- Did the planner struggle on this instance? Where did difficulty arise?
- How did the discrete battery encoding (number of battery levels, recharge model) affect the size of the search space and the search effort?
- Did the two domain models produce different results in plan length, plan cost, or runtime? What does this reveal about the relationship between modeling choices and planning efficiency?

A results table without this discussion will not receive full marks. This is where you demonstrate that you understand what the planner is doing and why, not just that you ran it.

---

8. Literature Connection

Your report must cite and briefly discuss **two published papers** related to the project. For each paper write a short paragraph of four to six sentences connecting it to a specific decision you made in your domain design or empirical evaluation. A generic summary of the paper without connection to your work is not sufficient.

Relevant topics include:

- UAV or drone mission planning with PDDL
- Resource-constrained planning (battery, fuel, payload)
- Multi-drone or multi-agent coordination in AI planning
- Heuristic search for logistics and delivery planning

Suggested starting points: Google Scholar, Semantic Scholar, and the ICAPS proceedings at https://ojs.aaai.org/index.php/ICAPS

---

9. Deliverables

Your ZIP file must contain:

File	Description
domain-A.pddl	First domain model
domain-B.pddl	Alternative domain model
problem-small.pddl	Small problem instance
problem-medium.pddl	Medium problem instance
problem-large.pddl	Large problem instance
problem-large-1drone.pddl	Single-drone variant of the large instance (Section 5)
validation/	Folder with VAL output files
README.txt	Exact commands used to run Fast Downward and VAL
report.pdf	Final report

10. Report Structure

Your report should be **8 to 12 pages**, excluding appendices, and follow this structure:

1. **Introduction** — motivation and scenario summary
2. **Problem analysis** — search space, types, predicates, actions, and design decisions
3. **PDDL implementation** — commented domain files with explanation of every predicate and action
4. **Alternative models** — description and structured comparison of Model A and Model B
5. **Problem instances** — description of each instance with location diagram

6. **Planner setup** — tools used, configurations tested, experimental procedure
7. **Results and analysis** — results table plus analytical discussion per instance
8. **Literature connection** — two paper discussions connected to specific project decisions
9. **Limitations and future work** — what the model does not capture and how it could be extended
10. **Appendix** — full PDDL source code, example generated plans, VAL output

---

11. Evaluation Criteria

Criterion	Points	What we look for
PDDL implementation	4	Both domain files are syntactically and semantically correct; the symbolic battery model is properly implemented; actions and predicates are well-designed and clearly commented in code and report
Problem instances and planner execution	2	Three instances scale meaningfully and meet their design constraints; both models run correctly on all instances; every reported plan is validated with VAL
Problem and results analysis	4	Alternative models are compared with evidence from experiments; analytical discussion per instance is substantive; literature connection is relevant and specific
Total	**10**	

12. Recommended Tools

Tool	Purpose	Link
Fast Downward	Planner	http://www.fast-downward.org
VAL	Plan validator	https://github.com/KCL-Planning/VAL
Planning.Domains	Online PDDL editor (for editing and syntax checks; report results from your own Fast Downward runs)	http://editor.planning.domains
PDDL extension for VS Code	Editor with syntax checks and quick planner runs (report results from your own Fast Downward runs)	VS Code Marketplace: “PDDL” by Jan Dolejsi

Setup: Fast Downward and VAL are built from source and are easiest to use on Linux or macOS. On Windows, use WSL 2 (Ubuntu) or Docker (Fast Downward provides an official Docker image, aibasel/downward). Check that both tools run on the lab examples before starting your experiments.

Set up the tools first. Install Fast Downward and VAL in the first two days after this statement is published and test them on a course example (for example the Blocks World of Lab 1) before you start modelling. With Docker, Fast Downward runs without building it (Linux/macOS/WSL shell, run in the folder with your files):

```
docker run --rm -v "$PWD":/work aibasel/downward \
    --plan-file /work/sas_plan --alias lama-first \
    /work/domain-A.pddl /work/problem-small.pddl
```

Build VAL with the instructions in its README (Linux, macOS or WSL 2).

Own work and AI tools. You may use AI assistants only for support (for example to check PDDL syntax); state in the report what you used them for. Every member of the group must be able to explain every predicate, action, instance and result. The

lecturer may ask any group for a short oral check (about 10 minutes) of its submission.

Questions about this assignment should be directed to Dr. Najeeb Jebreel, lecturer of Part I (Planning) (najeeb.jebreel@urv.cat). The lecturer of Part II (Reasoning) is Dr. Aida Valls, who is also the course coordinator (aida.valls@urv.cat). Groups wishing to propose a different alternative for the two-model requirement must obtain approval by 28/10/2026.